



MASTER D'EXCELLENCE INTELLIGENCE ARTIFICIELLE
TRAVAUX DIRIGÉS — MACHINE LEARNING

TD N° 02

Prétraitement des Données & Feature Engineering

Durée : 2 heures 30 minutes	Niveau : Débutant – Intermédiaire
Dataset : BankChurners (Kaggle)	Outils : Python, sklearn, pandas, seaborn

Objectifs du TD

- **Diagnostiquer** la qualité d'un dataset réel : types, manquants, outliers
- Comprendre les mécanismes **MCAR / MAR / MNAR** et choisir une stratégie adaptée
- **Comparer 4 stratégies d'imputation** et analyser leur impact sur la distribution
- **Encoder** correctement des variables binaires, ordinales et nominales
- Choisir entre **StandardScaler**, **RobustScaler** et **MinMaxScaler** selon le contexte
- Assembler un **ColumnTransformer** complet et réutilisable sans data leakage

Pr. BEN LAHMAR EL Habib · Semestre S2 2025–2026



Contexte et Mise en Situation

Vous travaillez pour une banque qui souhaite prédire l'attrition client (churn). Le dataset **BankChurners** contient 10 127 clients avec des caractéristiques démographiques et comportementales.

Avant toute modélisation, la réalité du terrain impose un nettoyage méticuleux des données. Ce TD vous guide à travers les étapes fondamentales du preprocessing, en vous faisant prendre des décisions justifiées à chaque étape.

Rappel important

Le dataset BankChurners original ne contient **aucune valeur manquante**. Dans ce TD, nous en simulons volontairement afin d'apprendre les techniques d'imputation dans un contexte contrôlé.

Description du Dataset BankChurners

Variable	Type	Exemple	Traitement requis
Attrition_Flag	Cible binaire	Existing Customer	Label Encoding → 0/1
Customer_Age	Num. continu	45	RobustScaler
Gender	Cat. binaire	M / F	Label Encoding
Education_Level	Cat. ordinale	Graduate	Ordinal Encoding (ordre logique)
Income_Category	Cat. ordinale	\$40K–\$60K	Ordinal Encoding (ordre revenu)
Card_Category	Cat. nominale	Blue / Gold	One-Hot Encoding
Total_Trans_Amt	Num. continu	4 756	RobustScaler (outliers)
Credit_Limit	Num. continu	12 691	RobustScaler (outliers)

Statistiques clés : 10 127 clients | 21 features | 16,1% de churn | 0 manquant natif



Partie 1 — Diagnostic et Simulation des Manquants

Durée estimée : 25 minutes | Un preprocessing aveugle est dangereux. Cette partie vous apprend à **écouter les données** avant de les transformer.

Q 1.1

Chargement et profil initial du dataset

Consigne

Chargez le dataset BankChurners, supprimez les 2 dernières colonnes (scores Naive Bayes inutiles) et produisez un profil complet.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Chargement du dataset
raw = pd.read_csv('BankChurners.csv')
data = raw[raw.columns[:-2]].copy() # supprimer 2 dernières colonnes

# — À COMPLÉTER —
print(f'Dimensions : {data.shape}')          # (10127, 19)
print(data.dtypes.value_counts())            # Combien de num. vs cat. ?

# Variables catégorielles et leurs valeurs uniques
cat_cols = data.select_dtypes(include='object').columns
for col in cat_cols:
    print(f'{col:35s} → {list(data[col].unique())}')

# Statistiques descriptives des numériques
data.describe().round(2)
```

Questions d'analyse

1. Combien de variables numériques et catégorielles le dataset contient-il ?
2. Quel est le ratio Existing / Attrited ? Le dataset est-il déséquilibré ? Qu'est-ce que cela implique pour le choix de la métrique d'évaluation (cf. TD 9) ?
3. Quelle variable numérique présente le plus grand écart entre valeur min et max ? Qu'est-ce que cela implique pour le choix du scaler ?



- `data['Attrition_Flag'].value_counts(normalize=True)` donne le ratio en %
- `data.describe()['max'] - data.describe()['min']` révèle les plages
- `Credit_Limit` et `Total_Trans_Amt` ont des plages très larges → candidats pour RobustScaler

Q 1.2

Simulation de valeurs manquantes (mécanisme MCAR)

Rappel théorique — Les 3 mécanismes de données manquantes

MCAR (Missing Completely At Random) : disparition totalement aléatoire — indépendante de toute variable. Ex : bug logiciel.

MAR (Missing At Random) : dépend d'autres variables observées. Ex : les femmes remplissent moins souvent le champ revenu.

MNAR (Missing Not At Random) : dépend de la valeur elle-même. Ex : les hauts revenus refusent de déclarer leur revenu.

Consigne

Introduisez des valeurs manquantes MCAR sur 3 colonnes selon les taux suivants :

- **Gender** : 8 % de manquants (variable binaire)
- **Education_Level** : 12 % de manquants (ordinaire multi-classes)
- **Total_Trans_Amt** : 5 % de manquants (numérique continu)

```
rng = np.random.default_rng(42) # Reproductibilité garantie
datam = data.copy()

missing_config = {
    'Gender': 0.08,
    'Education_Level': 0.12,
    'Total_Trans_Amt': 0.05,
}

for col, rate in missing_config.items():
    n_missing = int(len(datam) * rate)
    idx = rng.choice(datam.index, size=n_missing, replace=False)
    datam.loc[idx, col] = np.nan
    print(f'{col:25s} → {n_missing} NaN ({rate*100:.0f}%)')

# Visualisation du profil de manquants
missing = datam.isnull().sum()[lambda s: s > 0]
missing.plot(kind='barh', color=['#B07D3A', '#1A7A6E', '#5B3D8F'])
plt.title('Profil des valeurs manquantes simulées')
```



```
plt.show()
```

Questions d'analyse

4. Pourquoi utilise-t-on `replace=False` dans `rng.choice` ?
5. En quoi consiste le mécanisme MCAR ? Pourquoi est-il le plus simple à traiter ?
6. Donnez un exemple concret de mécanisme MNAR dans un contexte bancaire différent de celui donné dans le cours.

Q 1.3

Détection des outliers — méthode IQR

Consigne

Visualisez les distributions des features numériques clés et quantifiez les outliers. Cette analyse guidera votre choix de scaler en Partie 4.

```
num_key = ['Credit_Limit', 'Total_Trans_Amt',
           'Customer_Age', 'Total_Revolving_Bal',
           'Avg_Utilization_Ratio']

# Boxplots
fig, axes = plt.subplots(1, 5, figsize=(18, 5))
for ax, col in zip(axes, num_key):
    ax.boxplot(data[col].dropna(), patch_artist=True)
    ax.set_title(col, fontsize=9)
plt.tight_layout() ; plt.show()

# Quantification IQR
print(f'Variable:30s} {'N outliers':>12s} {'%':>8s}')
for col in num_key:
    q1, q3 = data[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    mask = (data[col] < q1 - 1.5*iqr) | (data[col] > q3 + 1.5*iqr)
    n = mask.sum()
    print(f'{col:30s} {n:>12d} {n/len(data)*100:>7.1f}%')
```

Questions d'analyse

7. Quelle(s) feature(s) ont le plus d'outliers ? Quel pourcentage représentent-ils ?
8. Expliquez pourquoi MinMaxScaler serait un mauvais choix pour Credit_Limit.
9. Quel scaler recommandez-vous pour les variables financières de ce dataset ? Justifiez.



🔍 Point critique — Data Leakage

Quelle que soit la méthode de normalisation choisie, le scaler doit être `fit()` uniquement sur le **train set**. Utiliser `fit_transform(X_train)` puis `transform(X_test)` uniquement. Fitter sur `X_test` = data leakage = simulation d'une connaissance du futur.



Partie 2 — Imputation des Valeurs Manquantes

Durée estimée : 40 minutes | Il n'existe pas de stratégie universelle. Le choix dépend du type de variable, du taux de manquants et du mécanisme sous-jacent.

Tableau comparatif des stratégies d'imputation

Stratégie	Formule	Avantages	Inconvénients
Moyenne	strategy='mean'	Simple, rapide	Sensible aux outliers, réduit la variance
Médiane	strategy='median'	Robuste aux outliers, préserve la distribution	Ignore les corrélations
Constante 0	fill_value=0	Interprétable si 0 a un sens	Crée des pics artificiels
Mode	strategy='most_frequent'	Naturel pour les catégorielles	Peut sur-représenter une classe
Constante 'Unknown'	fill_value='Unknown'	Préserve l'info de non-réponse	Ajoute une modalité artificielle

Q 2.1 Imputation numérique — 3 stratégies comparées sur Total_Trans_Amt

Consigne

Appliquez les stratégies mean, median et constant(0) sur Total_Trans_Amt. Comparez visuellement l'impact sur la distribution et les statistiques descriptives.

```
from sklearn.impute import SimpleImputer

col = 'Total_Trans_Amt'
strategies = {
    'Moyenne': SimpleImputer(strategy='mean'),
    'Médiane': SimpleImputer(strategy='median'),
    'Zéro': SimpleImputer(strategy='constant', fill_value=0),
}
data_imputed = {}

for name, imp in strategies.items():
    df_imp = datam.copy()
    df_imp[col] = imp.fit_transform(
        df_imp[col].values.reshape(-1, 1))
    data_imputed[name] = df_imp
```



```
# Visualisation : 4 histogrammes côte à côte
fig, axes = plt.subplots(1, 4, figsize=(18, 5), sharey=False)
axes[0].hist(datam[col].dropna(), bins=40, color='gray', alpha=.7)
axes[0].set_title('Original (avec NaN)')

colors = ['#B07D3A', '#1A7A6E', '#9C3D38']
for ax, (name, df_imp), c in zip(axes[1:], data_imputed.items(), colors):
    ax.hist(df_imp[col], bins=40, color=c, alpha=.7)
    ax.axvline(df_imp[col].mean(), color='red', ls='--', lw=1.5)
    ax.axvline(df_imp[col].median(), color='blue', ls='--', lw=1.5)
    ax.set_title(f'Imputation : {name}')
plt.tight_layout() ; plt.show()

# Comparaison statistique
print(f"{'Stratégie':15s} {'Moyenne':>10s} {'Médiane':>10s} {'Std':>10s}")
for name, df_imp in data_imputed.items():
    s = df_imp[col]
    print(f"{name:15s} {s.mean():>10.1f} {s.median():>10.1f} {s.std():>10.1f}")
```

Questions d'analyse

10. Observez les histogrammes. Quelle stratégie préserve le mieux la forme de la distribution originale ?
11. Pour une variable financière asymétrique avec des outliers comme Total_Trans_Amt, laquelle de ces stratégies recommandez-vous ? Pourquoi ?
12. Que se passe-t-il sur l'histogramme avec l'imputation par zéro ? En quoi est-ce problématique ?

📖 Indice

- Comparez les écarts-types : l'imputation par la moyenne ou la médiane **réduit artificiellement la variance**
- Sur des données asymétriques, la médiane est moins affectée par les valeurs extrêmes
- Un pic artificiel à 0 crée une fausse modalité qui peut tromper le modèle

Q 2.2

Imputation catégorielle — Mode vs Constante 'Unknown'

```
for col in ['Gender', 'Education_Level']:
    # Stratégie 1 : Mode
    df_mode = datam.copy()
    imp_mode = SimpleImputer(strategy='most_frequent')
    df_mode[col] = imp_mode.fit_transform(
        df_mode[col].values.reshape(-1,1)).ravel()

    # Stratégie 2 : Constante 'Unknown'
```



```
df_unk = datam.copy()
imp_unk = SimpleImputer(strategy='constant', fill_value='Unknown')
df_unk[col] = imp_unk.fit_transform(
    df_unk[col].values.reshape(-1,1)).ravel()

# Visualisation comparative
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
datam[col].value_counts().plot(kind='bar', ax=axes[0], title='Original')
df_mode[col].value_counts().plot(kind='bar', ax=axes[1], title='Mode')
df_unk[col].value_counts().plot(kind='bar', ax=axes[2], title='Unknown')
plt.suptitle(f'Imputation de {col}', fontweight='bold')
plt.tight_layout() ; plt.show()
```

⚠ Attention

Le dataset BankChurners contient déjà **17,5 % de valeurs 'Unknown'** pour Education_Level. En choisissant la constante 'Unknown', vous fusionnez les manquants simulés avec les vrais Unknown. Est-ce souhaitable ? Discutez.

Questions d'analyse

13. Pour Gender (variable binaire, répartition 53/47), quelle stratégie choisissez-vous ? Justifiez.
14. Pour Education_Level qui contient déjà des 'Unknown', quelle stratégie est sémantiquement cohérente ?
15. Quel piège technique se produit si une valeur n'est pas dans votre mapping manuel après l'encodage ?



Partie 3 — Encodage des Variables Catégorielles

Durée estimée : 30 minutes | Encoder une variable catégorielle exige de respecter sa sémantique : ordinaire $\neq$ nominale.

Q 3.1

Label Encoding — variables binaires et variable cible

Règle de base

- Variables **binaires** (2 modalités) → Label Encoding avec mapping explicite
- Variable **cible** → mapping manuel 0/1 (Attrited = 1 = classe positive)
- Toujours **vérifier** qu'aucun NaN n'est introduit par le mapping

```
df_enc = data.copy()

# 1. Variable CIBLE (Attrited = 1 = classe positive)
df_enc['Attrition_Flag'] = df_enc['Attrition_Flag'].map({
    'Existing Customer': 0,
    'Attrited Customer': 1
})

# 2. Gender
df_enc['Gender'] = df_enc['Gender'].map({'M': 1, 'F': 0})

# 3. Marital_Status
df_enc['Marital_Status'] = df_enc['Marital_Status'].map({
    'Single': 0, 'Married': 1, 'Divorced': 2, 'Unknown': -1
})

# — VÉRIFICATION OBLIGATOIRE —
check = ['Attrition_Flag', 'Gender', 'Marital_Status']
print('NaN introduits :', df_enc[check].isna().sum().sum())
# Si > 0 → une valeur du dataset n'est pas dans le mapping !
```

Questions d'analyse

16. Pourquoi est-il important que 'Attrited Customer' soit encodé 1 (et non 0) ?
17. Que se passe-t-il si vous utilisez LabelEncoder de sklearn sur la variable cible au lieu d'un mapping manuel ? Quel risque y a-t-il ?

Q 3.2

Encodage ordinal — Education_Level et Income_Category



Pourquoi l'ordre est crucial

Un encodage arbitraire (Uneducated=4, Doctorate=1) apprendrait au modèle une relation **inversée**. Pour Education_Level, l'ordre logique croissant est : Unknown → Uneducated → High School → College → Graduate → Post-Graduate → Doctorate.

```
# Ordre logique croissant pour Education
education_order = ['Unknown', 'Uneducated', 'High School',
                  'College', 'Graduate', 'Post-Graduate', 'Doctorate']

# Ordre logique croissant pour Income
income_order = ['Unknown', 'Less than $40K', '$40K - $60K',
               '$60K - $80K', '$80K - $120K', '$120K +']

# Mapping explicite (plus lisible qu'OrdinalEncoder ici)
edu_map = {v: i for i, v in enumerate(education_order)}
income_map = {v: i for i, v in enumerate(income_order)}

df_enc['Education_Level'] = df_enc['Education_Level'].map(edu_map)
df_enc['Income_Category'] = df_enc['Income_Category'].map(income_map)

# — VALIDATION : le taux de churn doit suivre l'ordre —
churn_by_edu = df_enc.groupby('Education_Level')['Attrition_Flag'].mean() * 100
churn_by_edu.plot(kind='bar', color='#5B3D8F', alpha=.8,
                  title='Taux de churn par niveau Education (encodé)')
plt.ylabel('Churn rate (%)') ; plt.show()
```

Questions d'analyse

18. Vérifiez visuellement que le taux de churn suit une tendance cohérente avec l'ordre d'encodage. Si ce n'est pas parfaitement monotone, est-ce un problème ? Pourquoi ?
19. Quelle est la différence entre l'encodage ordinal et le LabelEncoder de sklearn ? Quel est le risque du LabelEncoder sur une variable ordinale ?

Q 3.3

One-Hot Encoding — Card_Category (variable nominale)

Le Dummy Trap — Qu'est-ce que c'est ?

Avec 4 catégories (Blue, Silver, Gold, Platinum), 4 colonnes OHE sont **redondantes** : si Blue=Silver=Gold=0, alors Platinum=1 forcément. Cette multicollinéarité parfaite pose problème pour la Régression Logistique. Avec **drop_first=True**, on supprime la colonne de référence (Blue), réduisant à 3 colonnes suffisantes.



```
print('Distribution Card_Category :')
print(data['Card_Category'].value_counts())

# OHE avec drop_first (recommandé pour éviter le dummy trap)
ohe = pd.get_dummies(df_enc['Card_Category'],
                    prefix='card',
                    drop_first=True,    # supprime 'card_Blue'
                    dtype=int)

print(f'Nouvelles colonnes : {list(ohe.columns)}')
print(ohe.head())

# Intégration dans le DataFrame
df_enc = pd.concat([df_enc.drop('Card_Category', axis=1), ohe], axis=1)
print(f'DataFrame final : {df_enc.shape[1]} colonnes')
```

Questions d'analyse

20. Pourquoi utiliser `drop_first=True` ? Dans quel cas peut-on ne pas l'utiliser ?
21. Pourquoi `Card_Category` doit-elle être encodée en OHE plutôt qu'en ordinal ? Qu'est-ce qui différencie une variable ordinale d'une nominale ?



Partie 4 — Normalisation et Pipeline Final

Durée estimée : 35 minutes | L'étape finale : assembler toutes les transformations dans un ColumnTransformer réutilisable.

Comparaison des trois scalers

Scaler	Formule	Résistance outliers	Quand utiliser
StandardScaler	$(x - \mu) / \sigma$	Faible X	Distribution proche normale, peu d'outliers
RobustScaler	$(x - \text{médiane}) / \text{IQR}$	Élevée ✓✓	Variables financières, outliers fréquents
MinMaxScaler	$(x - \min) / (\max - \min)$	Nulle XX	Réseau de neurones, plage [0,1] requise

Q 4.1

Comparaison visuelle des trois scalers sur Credit_Limit

```
from sklearn.preprocessing import (StandardScaler,
                                    RobustScaler, MinMaxScaler)

col = 'Credit_Limit'
X_col = data[[col]].values

scalers = {
    'Original': data[col].values,
    'StandardScaler': StandardScaler().fit_transform(X_col).ravel(),
    'RobustScaler': RobustScaler().fit_transform(X_col).ravel(),
    'MinMaxScaler': MinMaxScaler().fit_transform(X_col).ravel(),
}

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes = axes.flatten()

for ax, (name, vals) in zip(axes, scalers.items()):
    ax.hist(vals, bins=50, alpha=.75, edgecolor='white')
    ax.axvline(vals.mean(), color='red', ls='--', lw=2)
    ax.axvline(np.median(vals), color='blue', ls='--', lw=2)
    ax.set_title(f'{name} [min={vals.min():.1f}, max={vals.max():.1f}]')

plt.suptitle('Comparaison des scalers - Credit_Limit', fontweight='bold')
plt.tight_layout() ; plt.show()
```



Questions d'analyse

22. Observez la plage [min, max] de chaque scaler. Lequel écrase le moins les outliers ?

23. Sur ce dataset, quel scaler recommandez-vous pour les variables financières ? Justifiez en 2-3 phrases.

Q 4.2

ColumnTransformer — Pipeline de preprocessing complet

Objectif de cette question

Assembler un **ColumnTransformer** qui applique le bon traitement à chaque type de colonne, en une seule fois, sans data leakage. Cet objet sera réutilisé dans tous les TDs suivants.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import (RobustScaler, OrdinalEncoder,
                                   OneHotEncoder)
from sklearn.model_selection import train_test_split

# — Définition des colonnes par type —————
num_cols = ['Customer_Age', 'Credit_Limit', 'Total_Trans_Amt',
            'Total_Revolving_Bal', 'Avg_Utilization_Ratio',
            'Months_on_book', 'Total_Trans_Ct']

ord_cols = ['Education_Level', 'Income_Category']
nom_cols = ['Card_Category']
bin_cols = ['Gender', 'Marital_Status']

# — Split AVANT de fitter (anti-leakage !) —————
y = data['Attrition_Flag'].map({'Existing Customer':0,
                               'Attrited Customer':1})
X = data.drop('Attrition_Flag', axis=1)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# — Sous-pipelines —————
num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', RobustScaler()),
])

ord_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='constant',
                              fill_value='Unknown')),
    ('encoder', OrdinalEncoder(categories=[
        ['Unknown', 'Uneducated', 'High School', 'College',
         'Graduate', 'Post-Graduate', 'Doctorate'],
        ['Unknown', 'Less than $40K', '$40K - $60K',
```



```
'$60K - $80K', '$80K - $120K', '$120K +']
], handle_unknown='use_encoded_value', unknown_value=-1)),
])

nom_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first', sparse_output=False,
                              handle_unknown='ignore')),
])

bin_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder()),
])

# — Assemblage final —————
preprocessor = ColumnTransformer(transformers=[
    ('num', num_pipe, num_cols),
    ('ord', ord_pipe, ord_cols),
    ('nom', nom_pipe, nom_cols),
    ('bin', bin_pipe, bin_cols),
], remainder='drop')

# Fit sur train UNIQUEMENT, transform sur train et test
X_train_pp = preprocessor.fit_transform(X_train)
X_test_pp = preprocessor.transform(X_test)

print(f'X_train préprocessé : {X_train_pp.shape}')
print(f'X_test préprocessé : {X_test_pp.shape}')
print(f'NaN restants : {np.isnan(X_train_pp).sum()}')
```

Questions d'analyse

24. Pourquoi effectue-t-on le `train_test_split` AVANT de créer le `ColumnTransformer` ?
25. Expliquez la différence entre `fit_transform(X_train)` et `transform(X_test)`. Que calcule concrètement `fit()` pour le `RobustScaler` ?
26. Comment récupérez-vous la médiane utilisée par le `RobustScaler` pour vérifier qu'il n'a été fitté que sur le train set ?

✓ Sauvegarde pour les prochains TDs

```
import joblib
joblib.dump(preprocessor, 'churn_preprocessor.pkl')
joblib.dump((X_train_pp, X_test_pp, y_train, y_test),
            'churn_splits.pkl')
print('Pipeline sauvegardé – réutilisable à partir du TD 4')
```





Section Bonus — Pour aller plus loin

Ces exercices sont optionnels et ne seront pas évalués. Ils permettent d'approfondir des notions utiles pour les TDs avancés.

Bonus B1 — Feature Engineering : créer de nouvelles features

Créez 2 nouvelles features à partir des features existantes et évaluez si elles sont discriminantes pour prédire le churn.

```
df_feat = data.copy()

# Feature 1 : ratio transactions par mois de relation
df_feat['Trans_Per_Month'] = (df_feat['Total_Trans_Ct'] /
                              df_feat['Months_on_book'])

# Feature 2 : indicateur de 'carte endormie'
df_feat['Is_Dormant'] = (
    (df_feat['Avg_Utilization_Ratio'] == 0) &
    (df_feat['Total_Revolving_Bal'] == 0)
).astype(int)

# Corrélation avec la cible
y_bin = df_feat['Attrition_Flag'].map({'Existing Customer':0,
                                       'Attrited Customer':1})

for feat in ['Trans_Per_Month', 'Is_Dormant']:
    print(f'{feat:25s} → corr = {df_feat[feat].corr(y_bin):.3f}')
```

27. La feature `Trans_Per_Month` est-elle plus corrélée avec le churn que `Total_Trans_Ct` seule ? Pourquoi est-ce intéressant ?

28. Proposez une troisième feature engineered qui vous semble pertinente pour prédire le churn.

Bonus B2 — Comparaison avec IterativeImputer (MICE)

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# MICE : imputation multiple par équations chaînées
mice = IterativeImputer(max_iter=10, random_state=42)
X_mice = datam.select_dtypes(include=['number']).copy()
X_mice_imp = mice.fit_transform(X_mice)

# Comparer avec SimpleImputer(median)
print('MICE conserve la distribution :')
print(pd.DataFrame(X_mice_imp, columns=X_mice.columns))
```



```
['Total_Trans_Amt'].describe().round(2)
```



Livraison attendue

Ce que vous devez remettre

- Notebook **TD2_NomPrenom.ipynb** avec toutes les cellules exécutées et outputs visibles
- Chaque choix méthodologique **justifié en commentaire** (pourquoi médiane ? pourquoi RobustScaler ?)
- Au minimum **5 visualisations** : profil des manquants, comparaison des imputations, boxplots outliers, comparaison des scalers, distributions des encodages
- Une cellule Markdown finale de **synthèse** (~200 mots) : tableau récapitulatif des choix de preprocessing
- Fichier **churn_preprocessor.pkl** sauvegardé et joint au rendu — réutilisé au TD 4
- Délai : **avant le TD 3**, dépôt sur la plateforme pédagogique

Critères d'évaluation

Critère	Coefficient	Description
Qualité du diagnostic et des justifications	35%	Pertinence des choix argumentés
Exactitude technique des transformations	30%	Pipeline correct, absence de leakage
Visualisations et leur interprétation	25%	Clarté, titres, commentaires
Section Bonus (optionnel)	+10%	Feature engineering ou MICE

Ressources et Documentation

- **Documentation sklearn — Preprocessing** : <https://scikit-learn.org/stable/modules/preprocessing.html>
- **Documentation sklearn — Imputation** : <https://scikit-learn.org/stable/modules/impute.html>
- **pandas.get_dummies** : https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html
- **Dataset BankChurners (Kaggle)** : <https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers>
- **Tutoriel sklearn ColumnTransformer** : https://scikit-learn.org/stable/auto_examples/compose/plot_column_transformer_mixed_types.html



Installation rapide

```
# Environnement virtuel
python -m venv venv_master_ia
source venv_master_ia/bin/activate # Linux/Mac

# Dépendances
pip install scikit-learn pandas numpy matplotlib seaborn joblib

# Lancer Jupyter
jupyter notebook
```